MATRIX MULTIPLICATION

2024 FALL

JINHO LEE

ECE, SNU

Accelerated Intelligent Systems Lab.

CONTENT

- Warmup
  - Amdahl's Law
  - Feynman's Algorithm
  - Foster's Algorithm

AMDAHL'S LAW

- Q: How much speedup can you get by parallelizing an application with P cores? (Of course, the maximum is P)

- Amdahl: It's limited by the parallel portion

Gene Amdahl (1922-2015)

AMDAHL'S LAW

"

There's a hard limit on how much speedup you can get by parallelization

"

Gene Amdahl (1922-2015)

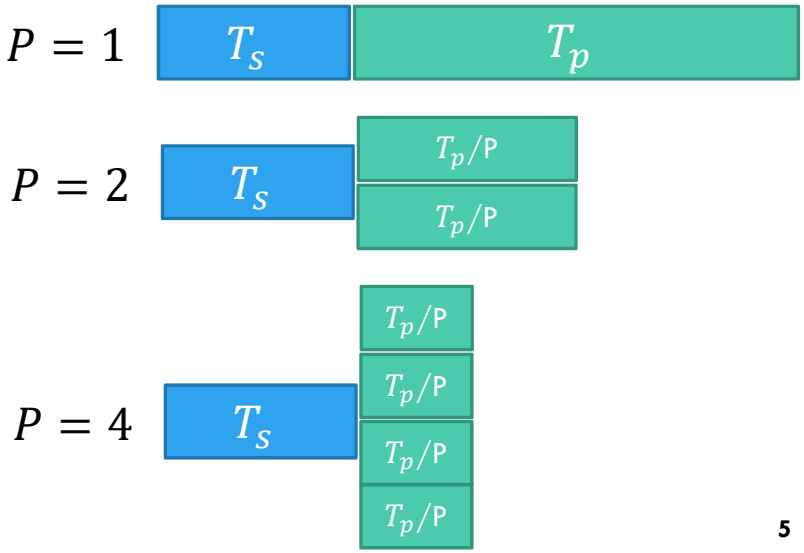
(this is not what he actually said)

AMDAHL'S LAW

- Assume your application is split into two parts:
  - Serial part (*S*) – cannot be parallelized
    - Atomic addition, dividing jobs, etc
  - Parallel part (*P*) – can be parallelized
    - Vector addition, matrix multiplication, etc

- $T(1) = T_s + T_p$ $\qquad T(P) = T_s + \frac{T_p}{P}$

- Speedup $S = \frac{T(1)}{T(P)} = \frac{T_s + T_p}{T_s + \frac{T_p}{P}}$



AMDAHL'S LAW

- $S = \frac{T(1)}{T(P)} = \frac{T_S + T_p}{T_S + \frac{T_p}{P}}$
- Setting f = fraction of the sequential portion
- $S = \frac{T(1)}{T(P)} = \frac{1}{f + \frac{1-f}{p}}$
- Amdahl's law:
  - $S(p) \leq \frac{1}{f + \frac{(1-f)}{p}}$
  - Meaning: you can only parallelize the parallelizable portion
    - Probably one of the most obvious laws in the world….
  - There's a limit in the parallelization

AMDAHL'S LAW

- Example
  - Your application is 40% sequential and 60% parallel.
  - With 6 processors, how much speedup can you get?
  - $S = \frac{T(1)}{T(P)} = \frac{1}{f + \frac{1-f}{p}}$
  - S = 1 / (0.4 + 0.6/6) = 2 (times)

AMDAHL'S LAW

- Example
  - Your application is 10% sequential and 90% parallel.
  - How many processors do you need to get 5x speedup?
  - $S = \frac{T(1)}{T(P)} = \frac{1}{f + \frac{1-f}{p}}$
  - f = 0.1
  - 5 = 1 / (0.1 + 0.9/P)
  - P = 9

AMDAHL'S LAW

- Example
  - Your application is 50% sequential and 50% parallel.
  - How many processors do you need to get 2x speedup?
  - $S = \frac{T(1)}{T(P)} = \frac{1}{f + \frac{1-f}{p}}$
  - f = 0.5
  - 2 = 1 / (0.5 + 0.5/P)
  - P = ∞ (you can't)

FEYNMAN ALGORITHM

- Suggested a general algorithm for solving any problem

1. Write down the problem.
2. Think real hard.
3. Write down the solution.

Richard Feynman (1918-1988)

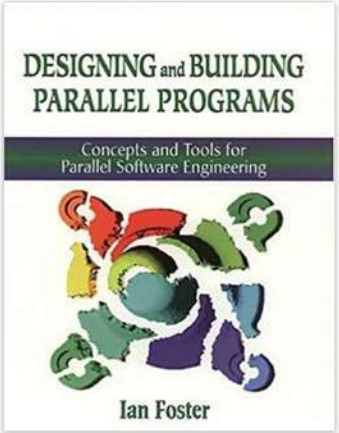
FOSTER'S METHODOLOGY

Ian Foster (1959-)

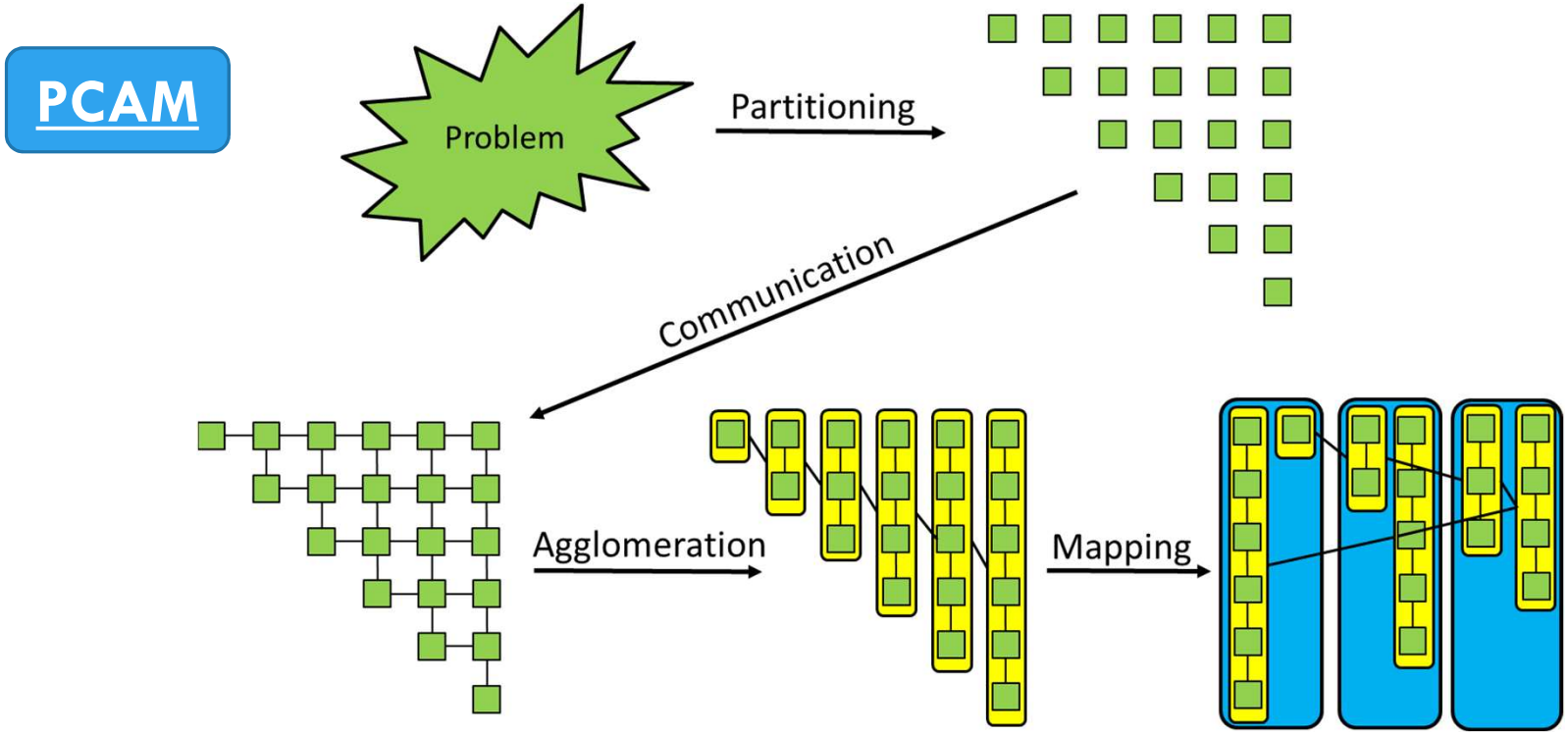
- Suggested a general algorithm for ~~solving~~ parallelizing any problem

PCAM

1. **Partitioning:** decompose the problem into a large amount of small tasks
2. **Communication:** determine the required communication between tasks
3. **Agglomeration:** combine identified tasks into larger tasks
4. **Mapping:** assign the agglomerated to processes/threads



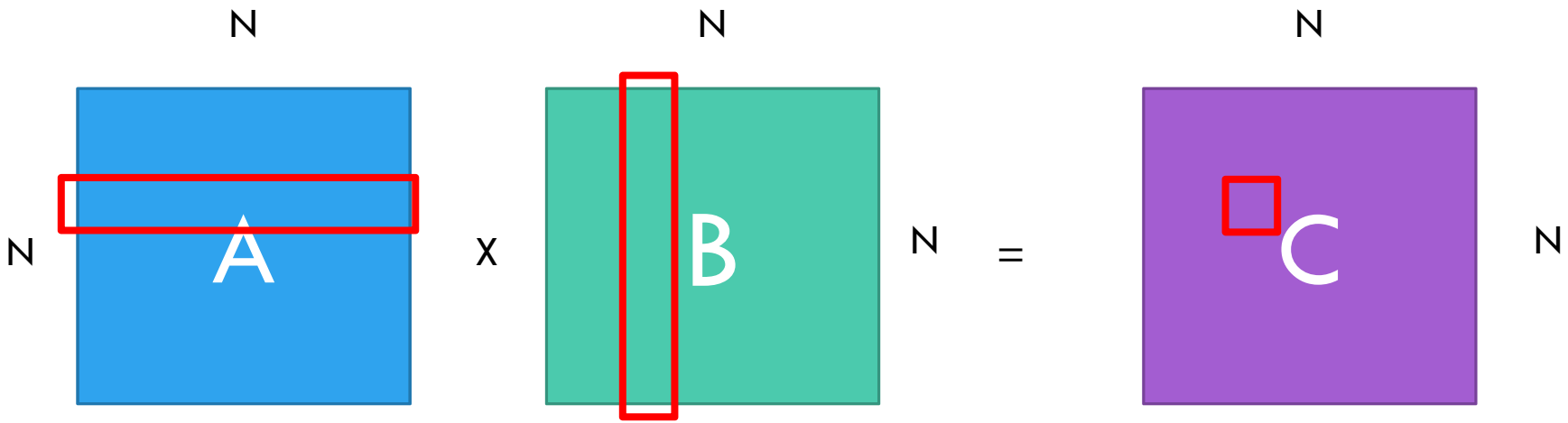
FOSTER`S PARALLEL ALGORITHM DESIGN METHOD



1. **Partitioning**: decompose the problem into a large amount of small (*Fine-grained*) tasks that can be executed in parallel
2. **Communication**: determine the required communication between tasks
3. **Agglomeration**: combine identified tasks into larger (*coarse-grained*) tasks in order to reduce communication by improving data locality
4. **Mapping**: assign the agglomerated to processes/threads in order to minimize communication, enable concurrency, and balance workload

https://docs.google.com/viewer?url=http://parallelprogrammingbook.org/slides/chapter_02.pptx#

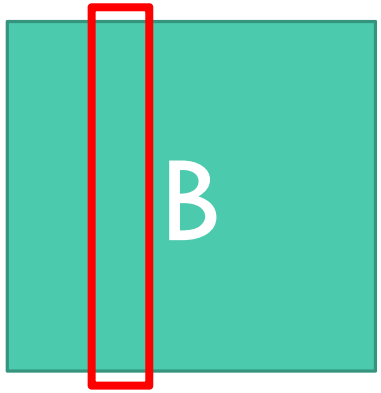
MATRIX MULTIPLICATION

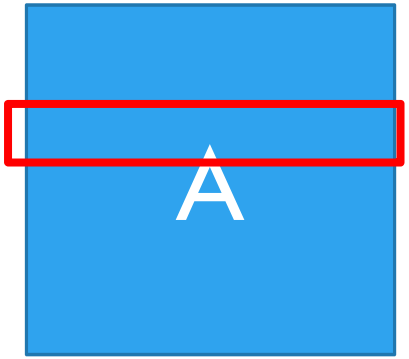


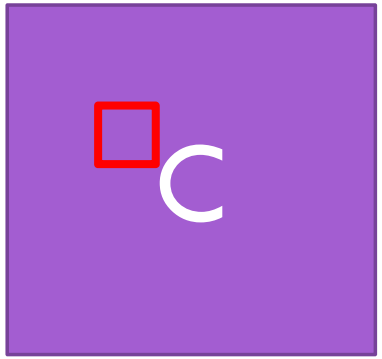
- $C_{ij} = \sum_k A_{ik} B_{kj}$
- There are million ways to write the code, even without parallelization
- …and million times more ways to parallelize them

MATRIX MULTIPLICATION

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```







- This is one (very simple) way.

SERIAL MATRIX MULTIPLICATION

```
class Matrix
{
    public:
    Matrix(int dim):dim(dim)
    {
        elem = new int*[dim];
        for(int i=0;i<dim;i++)    {
            elem[i] = new int[dim];
            for(int j=0;j<dim;j++) {
                elem[i][j] = 0;
            }
        }
    }

    Matrix& mult(const Matrix& B)
    {
        assert(dim == B.dim);
        Matrix *out = new Matrix(dim);
        for(int i=0;i<dim;i++) {
            for(int j=0;j<dim;j++) {
                for(int k=0;k<dim;k++) {
                    out->elem[i][j] += elem[i][k] * B.elem[k][j];
                }
            }
        }
        return *out;
    }
private:
        int dim;
        int** elem;
};
```

using 2D array for an easier example..
but usually 1D is faster

- Simple triple-loop
- How do we parallelize this?

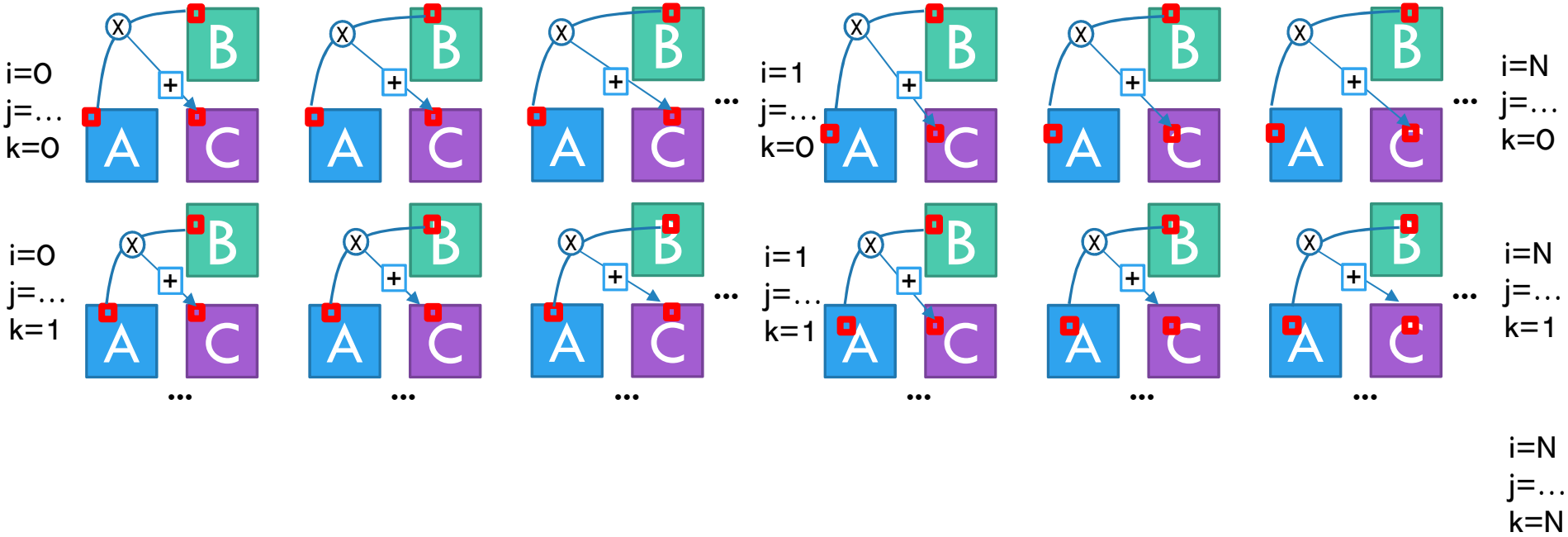
PARALLEL MATRIX MULTIPLICATION

- We don't really follow Foster's algorithm
- But Foster kinda tells us what to do

1. Partition the problem into many small pieces
2. Consider communication somehow
3. Combine them somehow

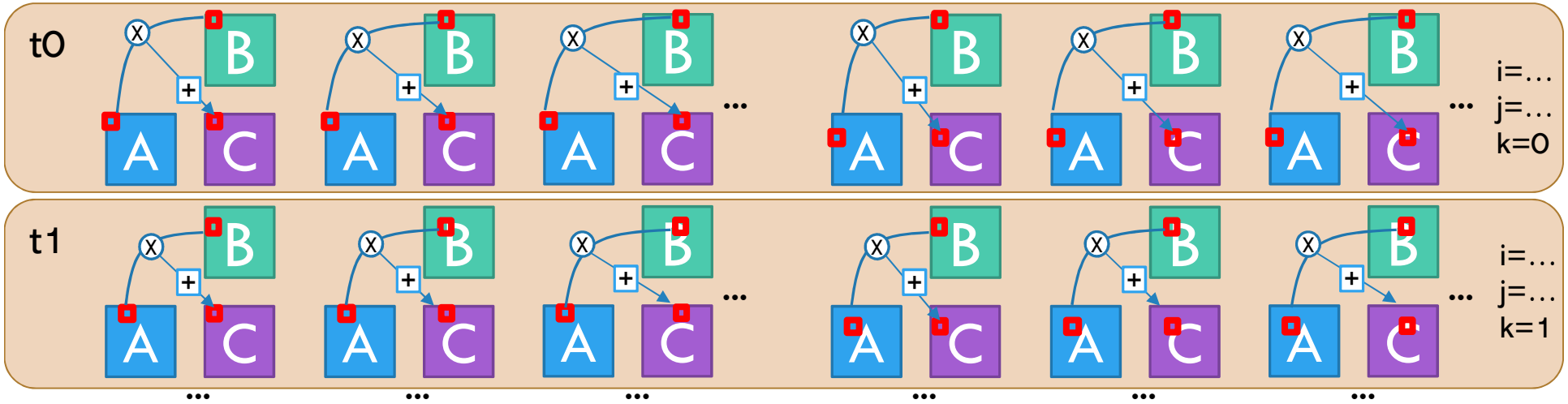
- We will start from a intentionally stupid algorithm and gradually build a better one

PARTITION THEM INTO SMALL TASKS



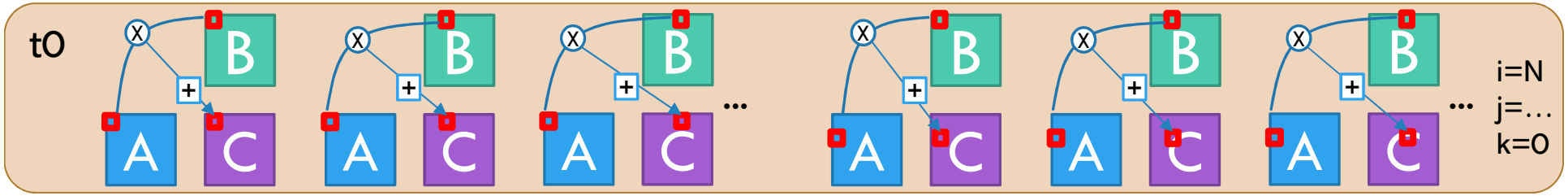
- Let's say this is how we partition matmul into small tasks
  - A task = MAC (Multiply-and-accumulate)
- There are N^3 tasks

PARTITION THEM INTO SMALL TASKS



- Let's combine them somehow
- Note: we do not want to use 1 thread for each task.
  - For a 800*800 matrix, you will need 512,000,000 threads
  - You don't have that much system resource
    - because each thread will consume some memory capacity
  - Too much communication/synchronization will harm your performance

PARTITION THEM INTO SMALL TASKS




```
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{
    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
}
```

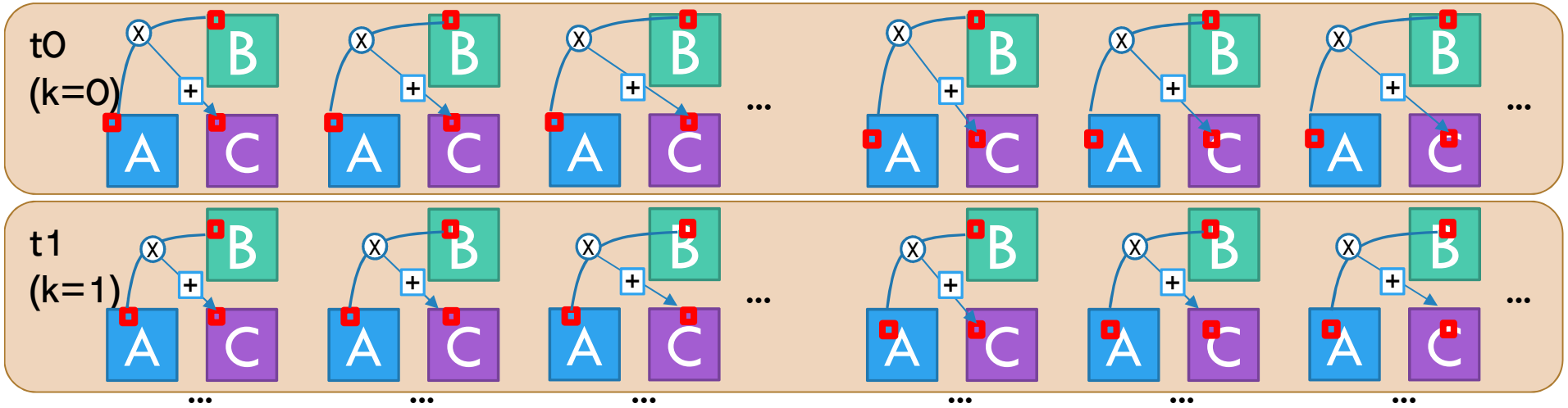
- After combining, we get a smaller job for each thread
  - Double loop instead of triple

PARALLEL MATRIX MULTIPLICATION

```
class Matrix
{
…
    friend void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k);
private:
        int dim;
        int** elem;
};

void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{

    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
}

int main()
{
    Matrix A(MATDIM);
    A.fillrand();
    Matrix B(MATDIM);
    B.fillrand();
    Matrix C(MATDIM);
    std::vector<std::thread> threads;
    for(int k=0;k<MATDIM;k++) {
        threads.push_back(
           std::thread(matmul_ij, std::ref(A), std::ref(B), std::ref(C), k));
    }

    for(auto& t:threads) {
        t.join();
    }
    return 0;
}
```

- Not the best code structure, but simple enough for us
- Any problem?

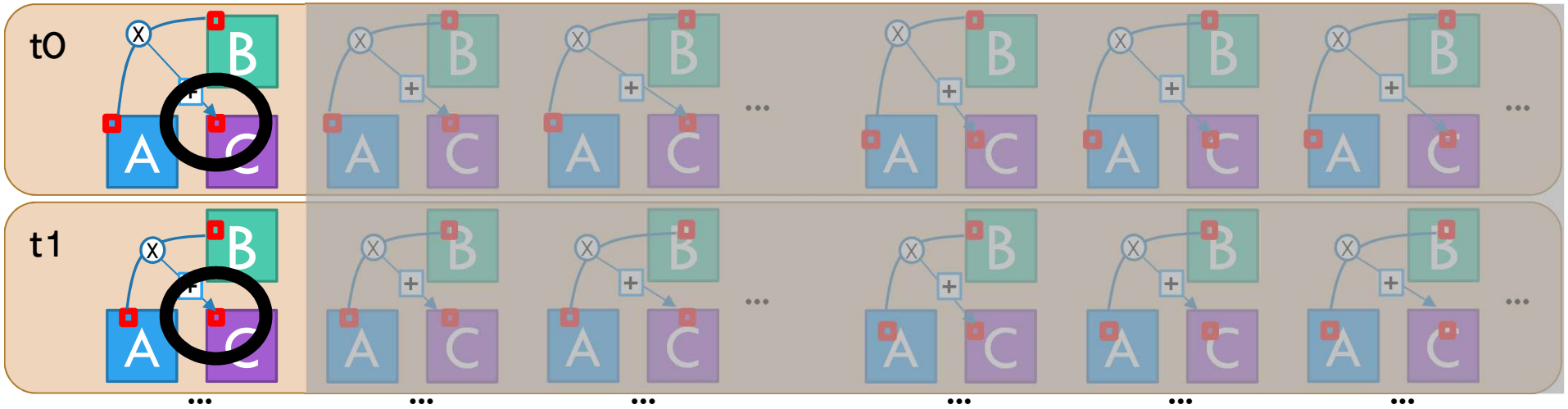
LOOK AT THE COMMUNICATION




```
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{

    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }

}
```

- Race condition!

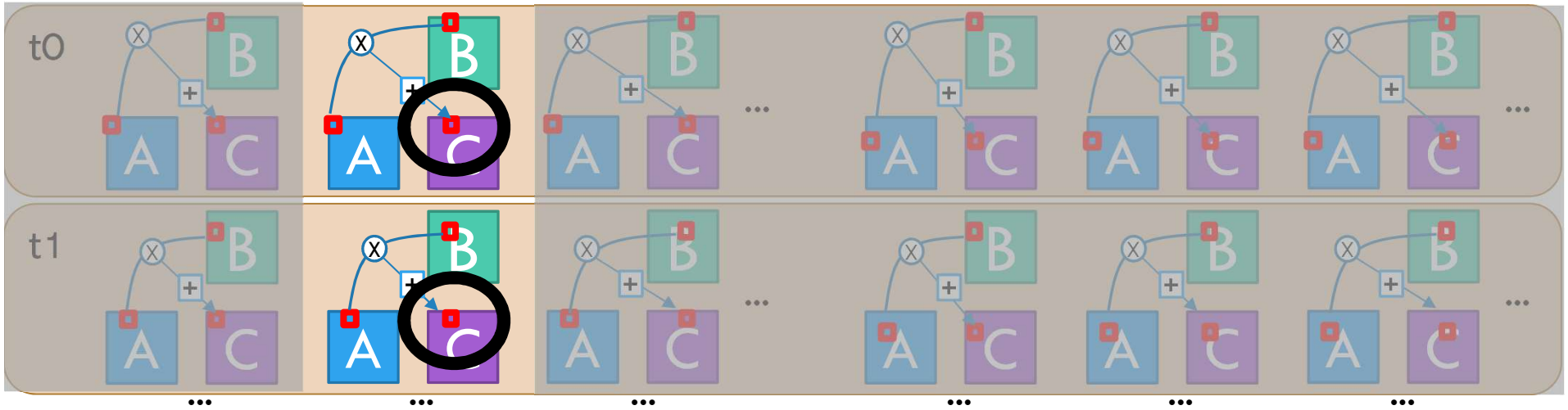
LOOK AT THE COMMUNICATION




```
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{

    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }

}
```

- Race condition!
  - between threads, on writing to matrix C

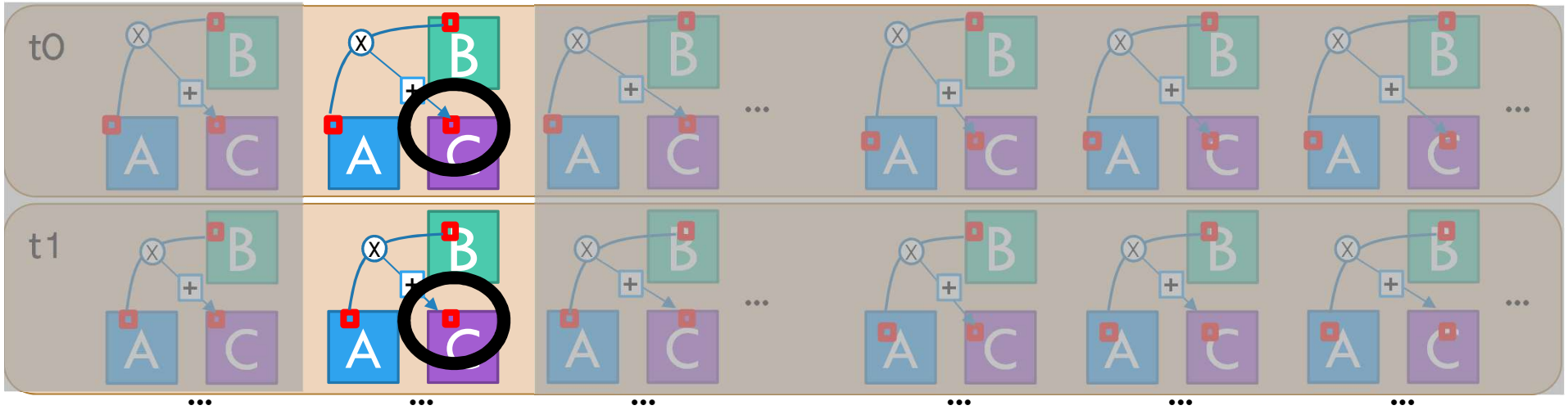
LOOK AT THE COMMUNICATION




```
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{

    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }

}
```

- Race condition!
  - between threads, on writing to matrix C

VERSION 1 – GLOBAL LOCKING




```
std::mutex m; //a global mutex
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{
    m.lock();
    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
    m.unlock();
}
```

- Adding mutex makes it work
  - Let's call it version 1

DOES IT RUN FASTER?

(Running on a Jetson Orin Nano)

Version	Time (800*800 Matrix)
Serial	1.09sec
Version1 (Global lock)	1.34sec

- It's slower!
- Mutex is making everything run serially
- Extra overhead for lock/unlock, thread switching

```
std::mutex m; //a global mutex
void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{
    m.lock();
    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
    m.unlock();
}
```

VERSION 2 – FINE-GRAINED LOCKING

```
class Matrix
{
    public:
    Matrix(int dim):dim(dim)
    {
        elem = new int*[dim];
        m = new std::mutex*[dim];
        for(int i=0;i<dim;i++)    {
            elem[i] = new int[dim];
            m[i] = new std::mutex[dim];
            for(int j=0;j<dim;j++) {
                elem[i][j] = 0;
            }
        }
    }
…
     private:
        int dim;
        int** elem;
        std::mutex** m;
};

void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{
    for(int i=0;i<A.dim;i++) {
        for(int j=0;j<A.dim;j++) {
            C.m[i][j].lock();
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
            C.m[i][j].unlock();
        }
    }
}
```

- Fine-grained locking
- Ensures parallel progress

VERSION 2

Version	Time (800*800 Matrix)
Serial	1.09sec
Version1 (Global lock)	1.34sec
Version2 (Fine-grain)	5.11sec

- It's even slower!
- Yes, it's running in parallel,
  but Lock & Unlock are very expensive operations..

+) For some systems, it can be marginally faster

- e.g., on my intel i7 desktop
- Depending on the cost of lock and unlock
  - Cost depends on hardware support and lock algorithm

VERSION 3 – COARSE-GRAINED LOCKING

```
class Matrix
{
    public:
    Matrix(int dim):dim(dim)
    {
        elem = new int*[dim];
        m = new std::mutex[dim];
        for(int i=0;i<dim;i++)    {
            elem[i] = new int[dim];
            m[i] = new std::mutex[dim];
            for(int j=0;j<dim;j++) {
                elem[i][j] = 0;
            }
        }
    }
…
     private:
        int dim;
        int** elem;
        std::mutex* m;//lock per C row
};

void matmul_ij(const Matrix& A, const Matrix& B, Matrix& C, int k)
{
    for(int i=0;i<A.dim;i++) {
        C.m[i].lock();
        for(int j=0;j<A.dim;j++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
        C.m[i].unlock();
    }
}
```

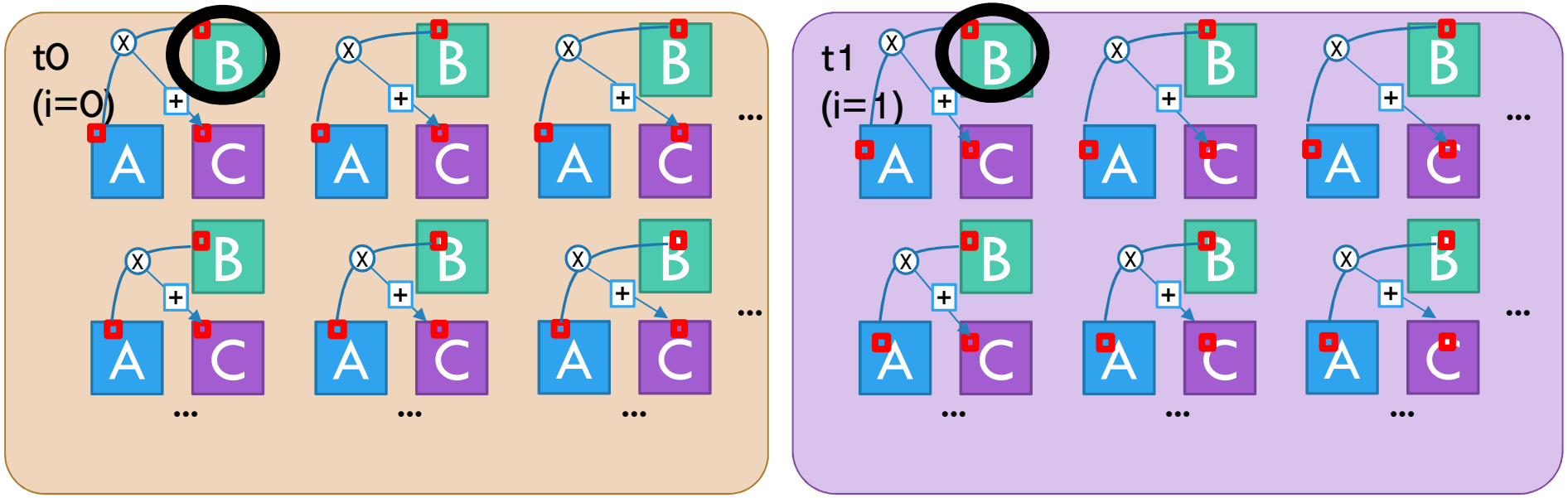
- We are having one lock per row
- Loose a little bit of parallelism
- But less frequent lock/unlock
- Can you find the sweet spot?

VERSION 3

Version	Time (800*800 Matrix)
Serial	1.09sec
Version1 (Global lock)	1.34sec
Version2 (Fine-grain)	5.11sec
Version3 (Coarse-grain)	0.28sec

- We got near-linear speedup!
  - 1.58 / 0.28 = 4.36x
  - Remember that we had 6 cores?

SORRY, THERE HAD BEEN A BETTER WAY ALL ALONG…

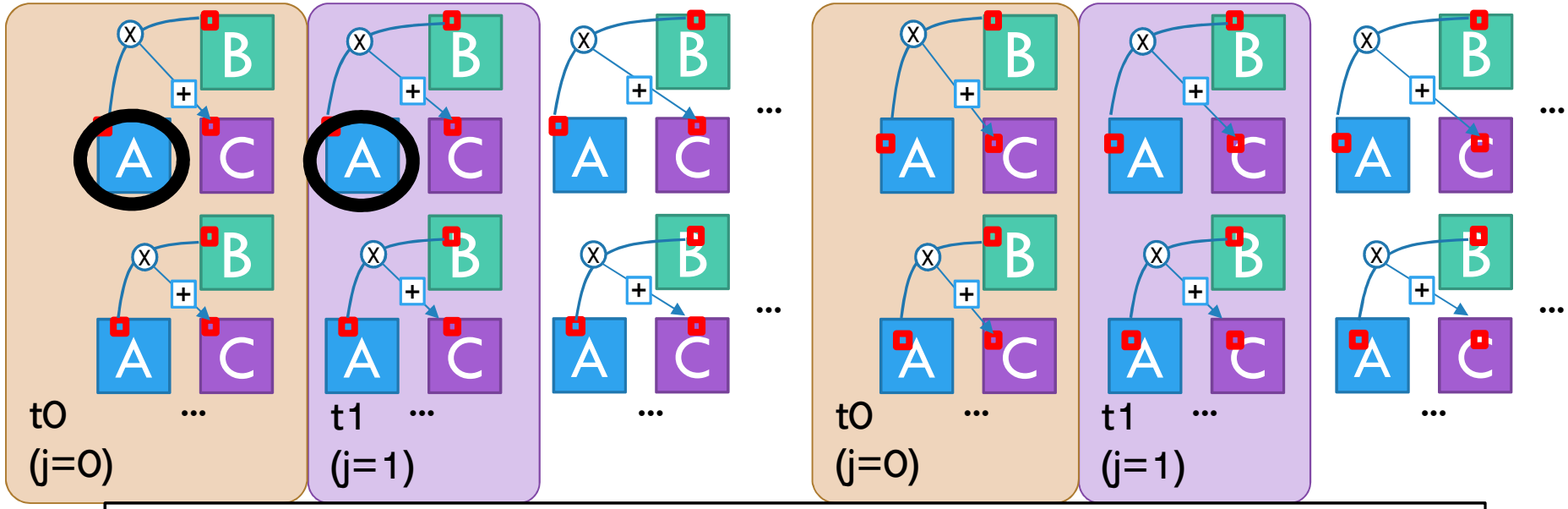



```
void matmul_jk(const Matrix& A, const Matrix& B, Matrix& C, int i)
{
    for(int j=0;j<A.dim;j++) {
        for(int k=0;k<A.dim;k++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
}
```

```
 std::vector<std::thread> threads;
        for(int i=0;i<MATDIM;i++) {
            threads.push_back(std::thread(matmul_jk, std::ref(A), std::ref(B), std::ref(D), i));
}
```

- If you partition/group it right, no race condition occurs
  - Reading same data from multiple threads is safe
- No locking needed at all

SORRY, THERE HAD BEEN A BETTER WAY ALL ALONG…




```
void matmul_ik(const Matrix& A, const Matrix& B, Matrix& C, int j)
{
    for(int i=0;i<A.dim;i++) {
        for(int k=0;k<A.dim;k++) {
            C.elem[i][j] += A.elem[i][k] * B.elem[k][j];
        }
    }
}
```

```
 std::vector<std::thread> threads;
        for(int j=0;j<MATDIM;j++) {
            threads.push_back(std::thread(matmul_ik, std::ref(A), std::ref(B), std::ref(D), j));
}
```

- If you partition/group it right, no race condition occurs
  - Reading same data from multiple threads is safe
- No locking needed at all

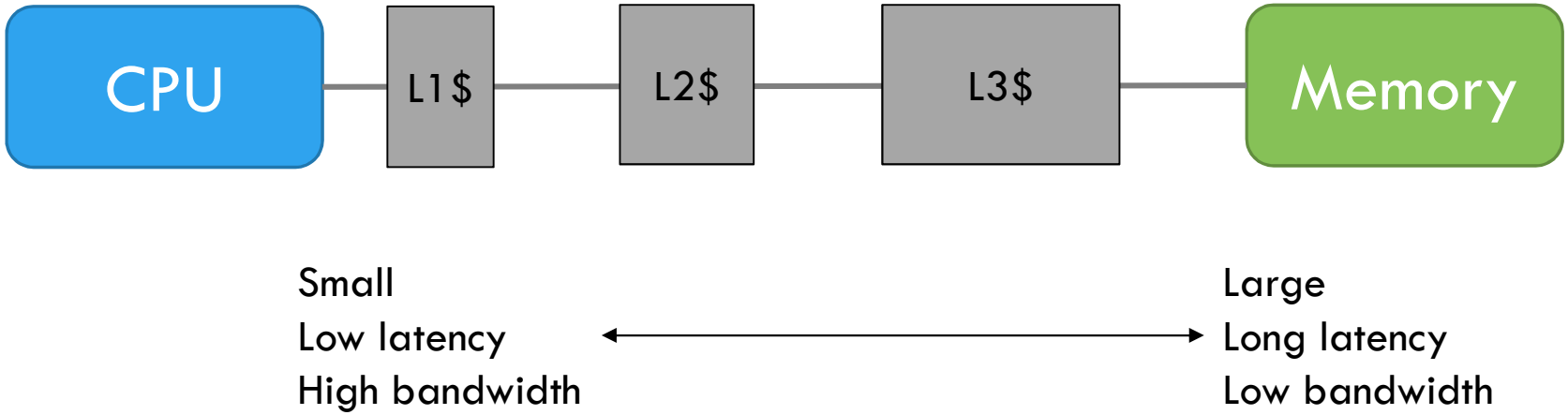
VERSION 4

Version	Time (800*800 Matrix)
Serial	1.09sec
Version1 (Global lock)	1.34sec
Version2 (Fine-grain)	5.11sec
Version3 (Coarse-grain)	0.28sec
Version4a (No lock-jk)	0.29sec
Version4b (No lock-ik)	0.19sec

- Similar or higher speedup, no lock at all
- Now the question is – why is the “ik” version faster?
- I will leave the detailed answer as a homework,
  but to tell you the principle – it’s the memory!

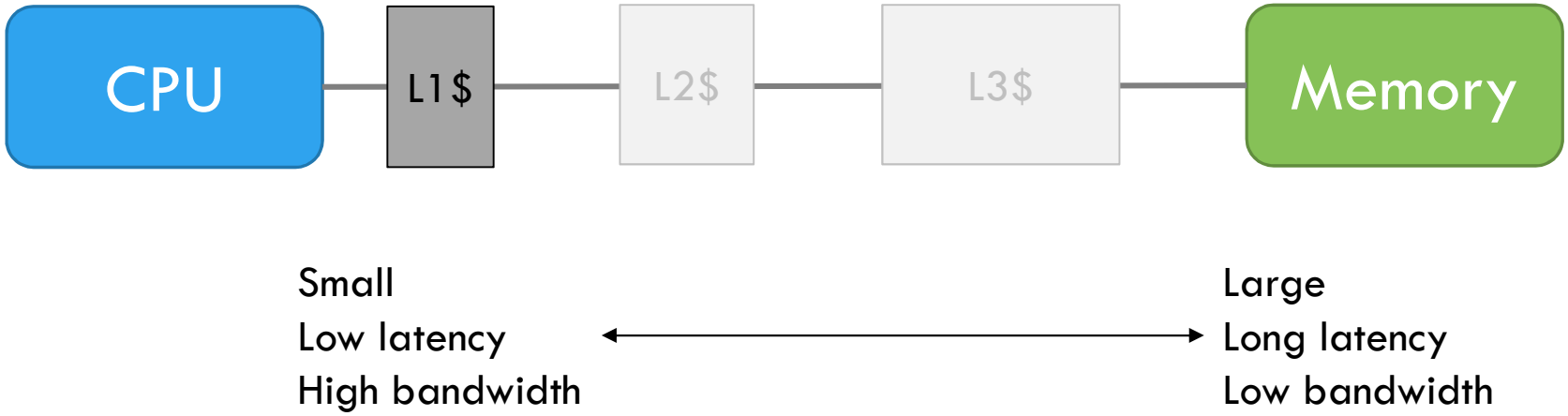
OPTIMIZATION FOR MEMORY

Consider Caches



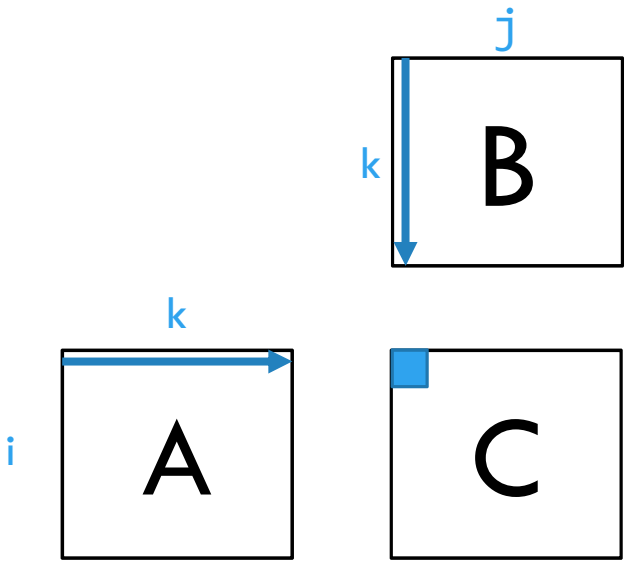
- L1 D-cache is about 32KB~64KB
  - = 8K~16K fp/int numbers
  - = can hold up to N=128 matrix at a time (128*128*4B = 64KB)
  - Practically, about N=64 matrix will be the maximum (store, A, B, C + others)
- From the cache you read 64B at a time (with 4B items, 16 elements)

Consider Caches



- L1 D-cache is about 32KB~64KB (64KB in our board)
  - = 8K~16K fp/int numbers
  - = can hold up to N=128 matrix at a time (128*128*4B = 64KB)
  - Practically, about N=64 matrix will be the maximum (store, A, B, C + others)
- From the cache you read 64B at a time (with 4B items, 16 elements)

Analyzing Cache Behavior on Matmul

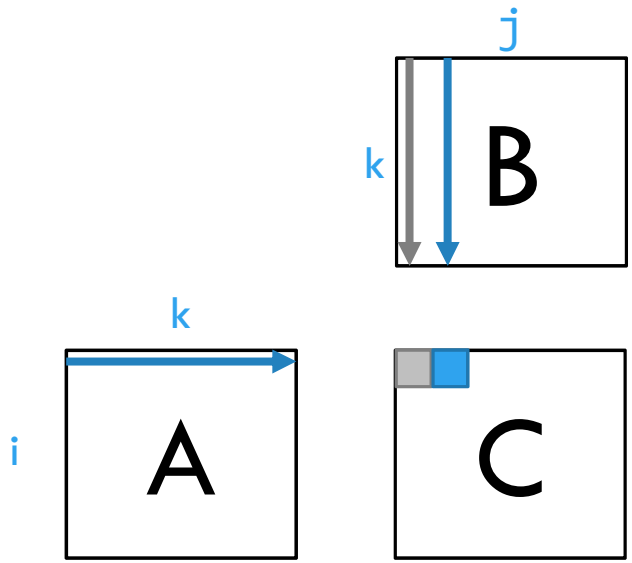



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	1
k	1→N

- All miss (cold miss)

Analyzing Cache Behavior on Matmul

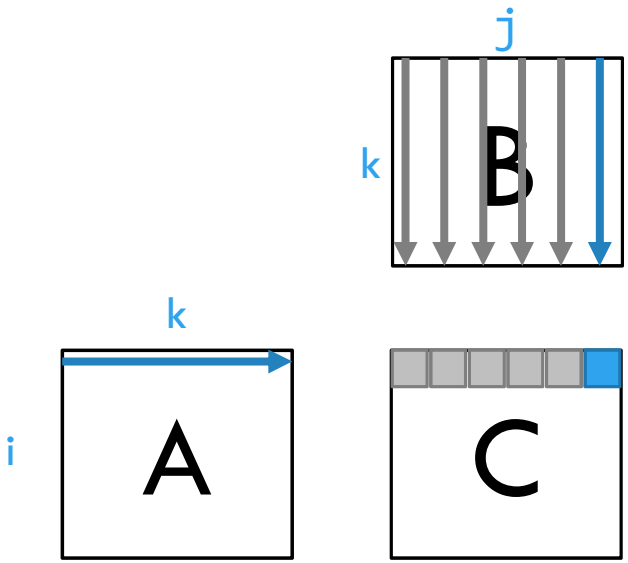



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	2
k	1→N

- A: all hit Let's assume N < Cache size
- B: all miss (cold miss)

Analyzing Cache Behavior on Matmul

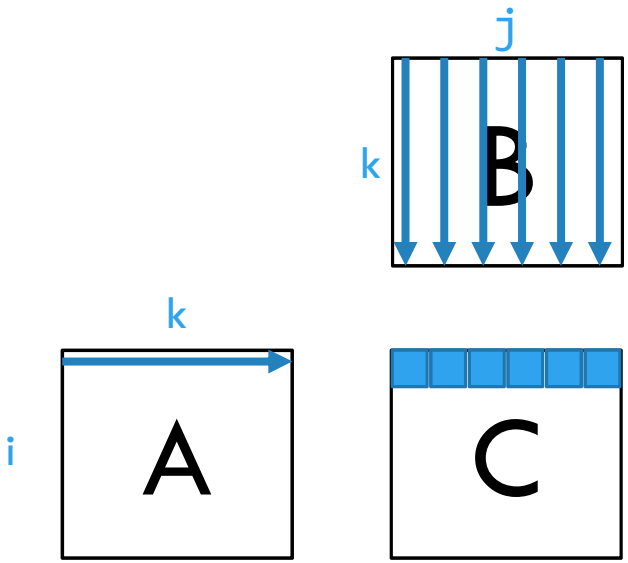



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	N
k	1→N

- A: all hit
- B: all miss (cold miss)

Analyzing Cache Behavior on Matmul

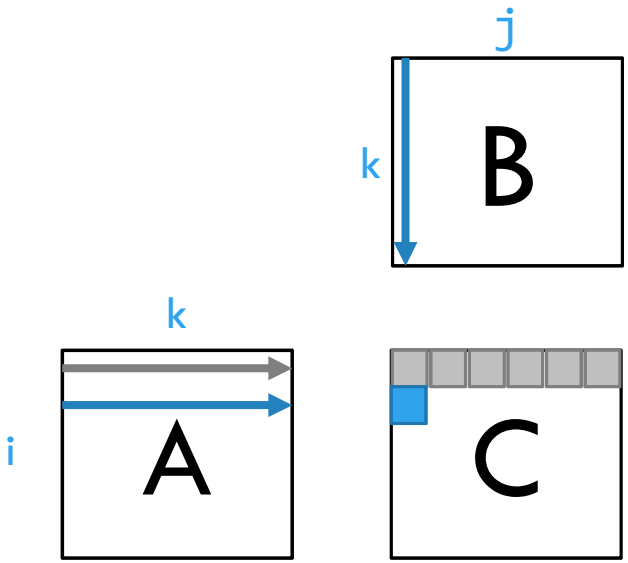



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	1→N
k	N*(1→N)

- A: N/c reads
- B: N^2/c reads
  - c: size of the cache line (16 here, assuming 4B integer and 64B cachelines)

Analyzing Cache Behavior on Matmul

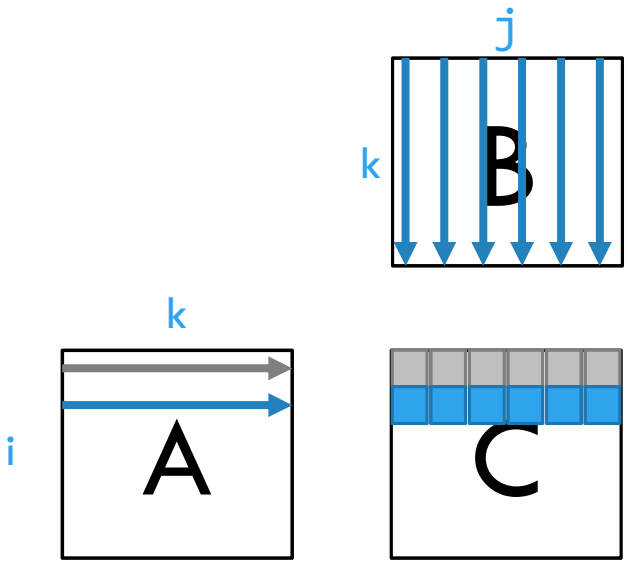



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	2
j	1
k	1→N

- A: miss (cold miss)
- B: miss (capacity miss)
  - Because B does not fit into L1 cache

Analyzing Cache Behavior on Matmul

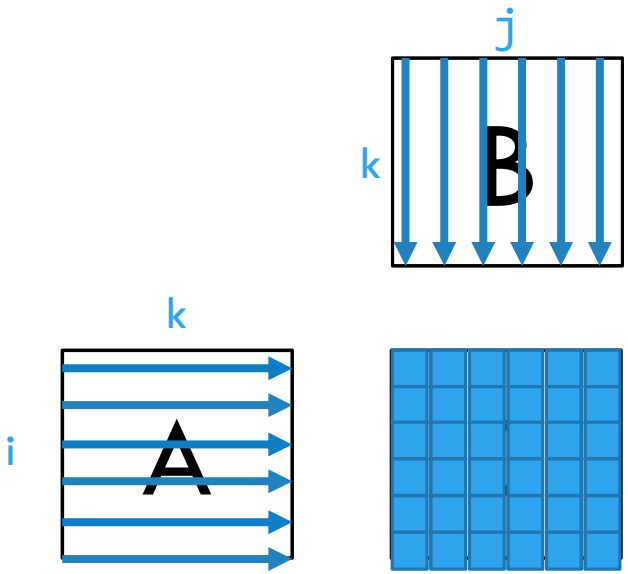



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	2
j	1→N
k	N*(1→N)

- A: N/c reads
- B: N^2/c reads

Analyzing Cache Behavior on Matmul

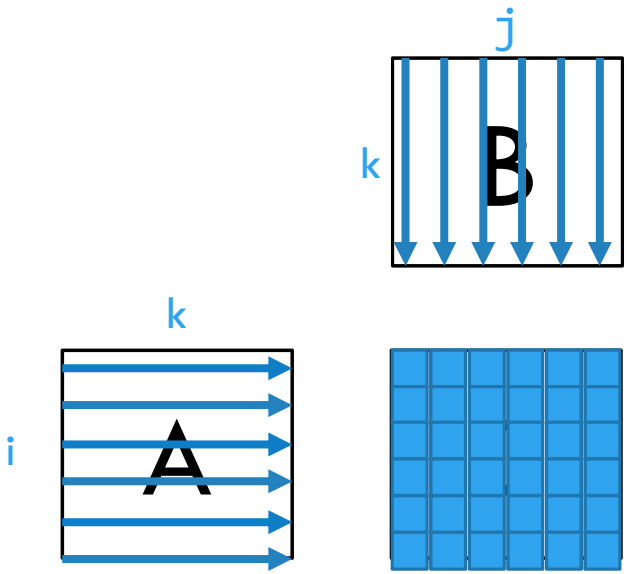



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1→N
j	N*(1→N)
k	N^2*(1→N)

- A: N^2/c reads
- B: N^3/c reads

Analyzing Cache Behavior on Matmul

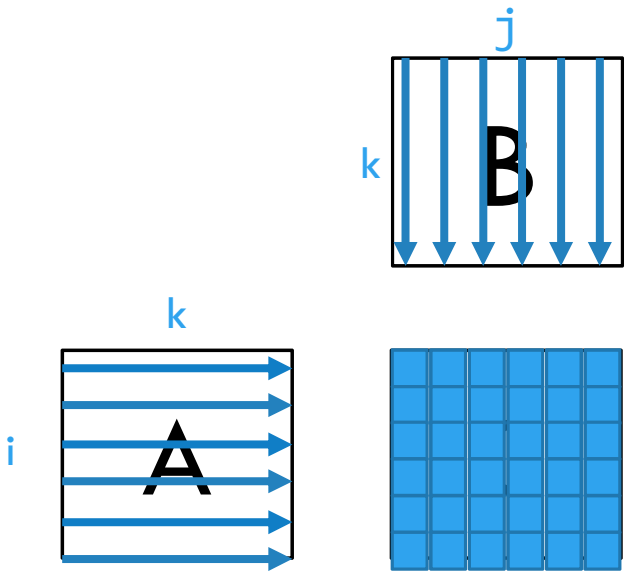



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1→N
j	N*(1→N)
k	N^2*(1→N)

- A: N^2/c reads Ideal. You have to read everything at least once
- B: N^3/c reads More than necessary
  Our goal is to reduce this
- C: N^2/c reads + N^2/c writes

THERE'S EVEN MORE



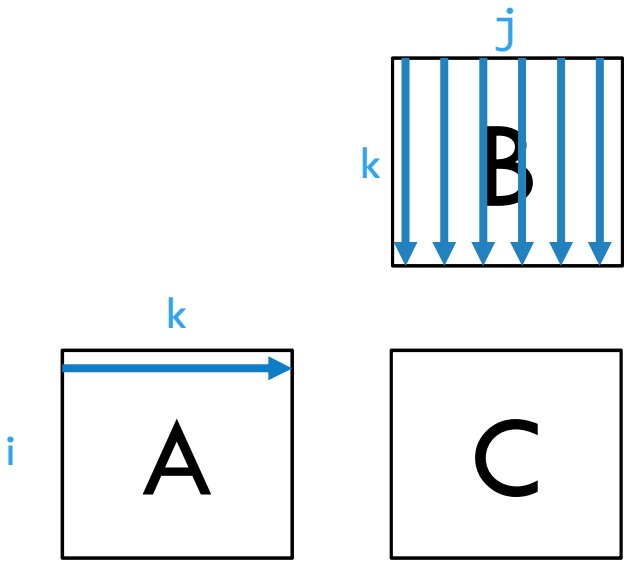

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1→N
j	N*(1→N)
k	N^2*(1→N)

- A: N^2/c reads
- B: N^3/c reads
- C: N^2/c reads + N^2/c writes

But before that, even getting here might not be that trivial

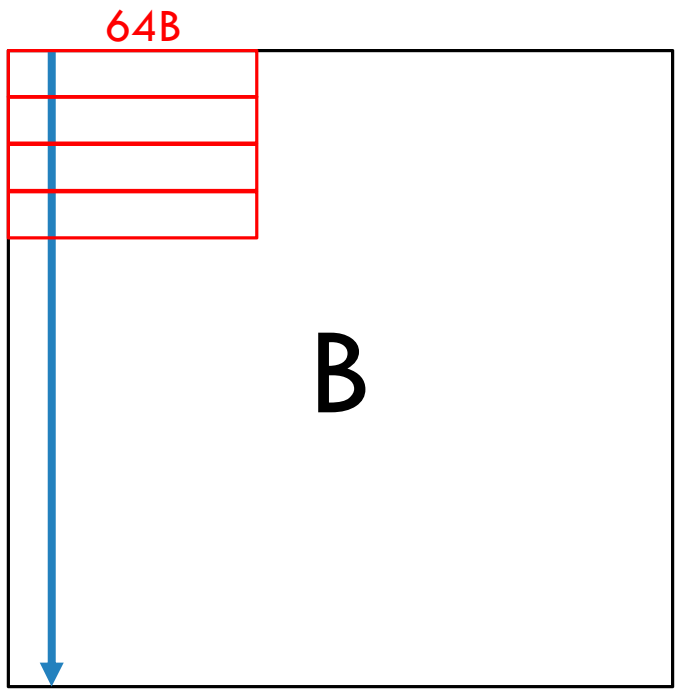
ACCESS PATTERN MISMATCH




```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

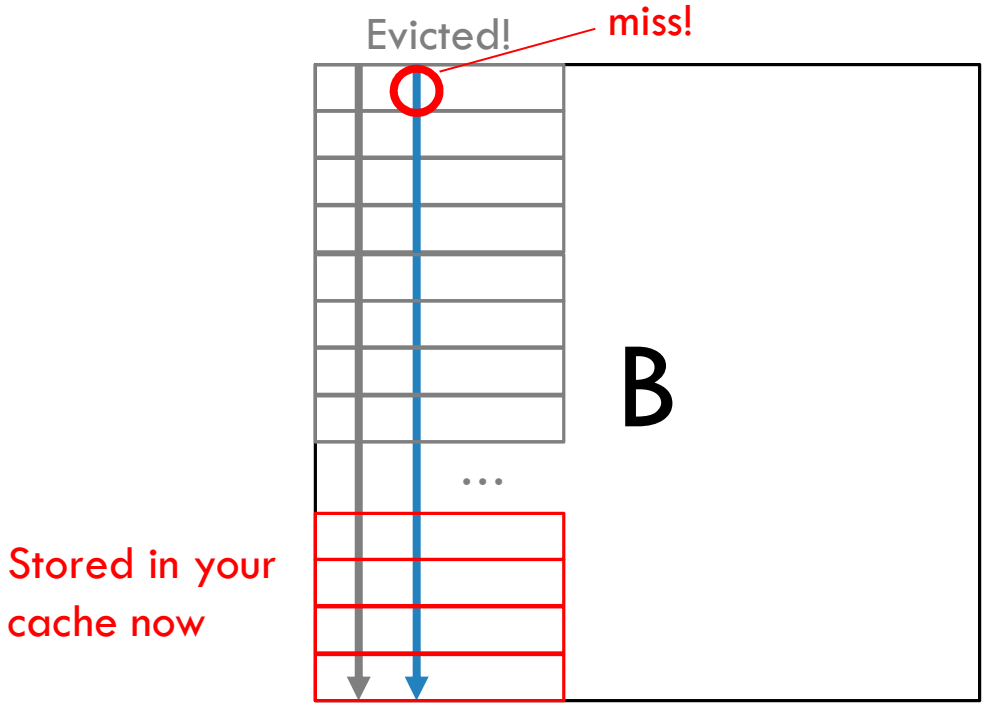
- We have been assuming that we need N^2/c reads to read entire B
- Is it true?

ACCESS PATTERN MISMATCH



- When you read B in a columnwise order, you have to read a cache line each time
- Your cache needs to store 16N instead of N

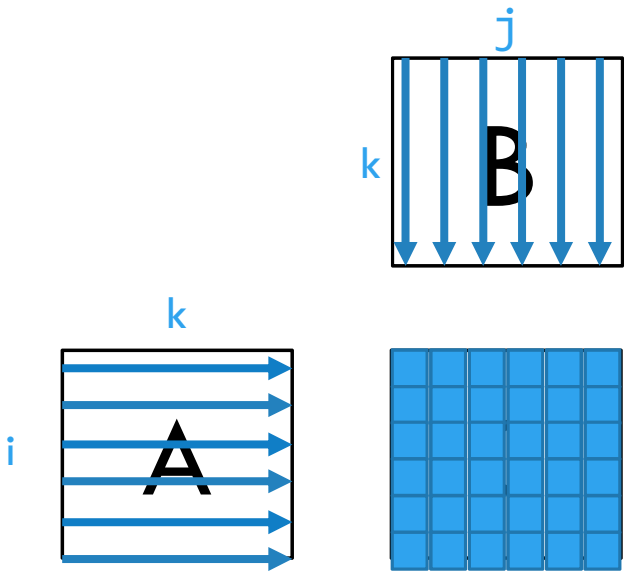
ACCESS PATTERN MISMATCH



- When you read B in a columnwise order, you have to read a cache line each time
- To have a cache hit on the first line, your cache needs to store 16N items
  - 16 items (of 4B) in a cache line

- If **cache size < 16N**, you get a miss at each read
- You end up reading **N^3** times instead of **N^3/c**

CACHE WAY CONFLICT




```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

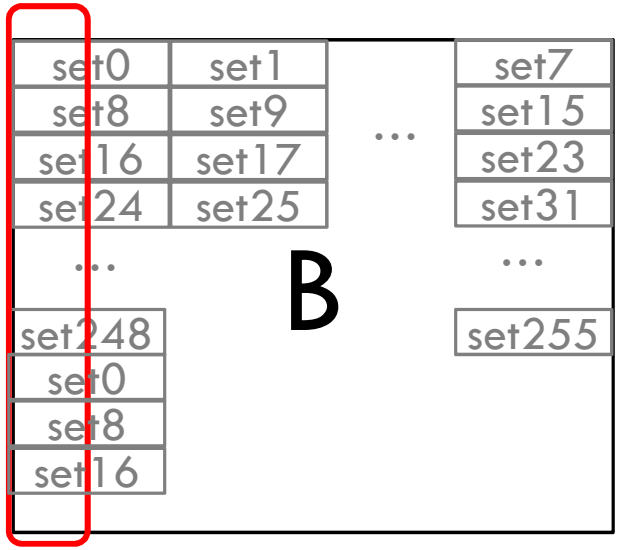
i	1→N
j	N*(1→N)
k	N^2*(1→N)

- A: N^2/c reads
- B: N^3/c reads (if 16N < cache size)
- C: N^2/c reads + N^2/c writes

However still,
We need more effort to get here

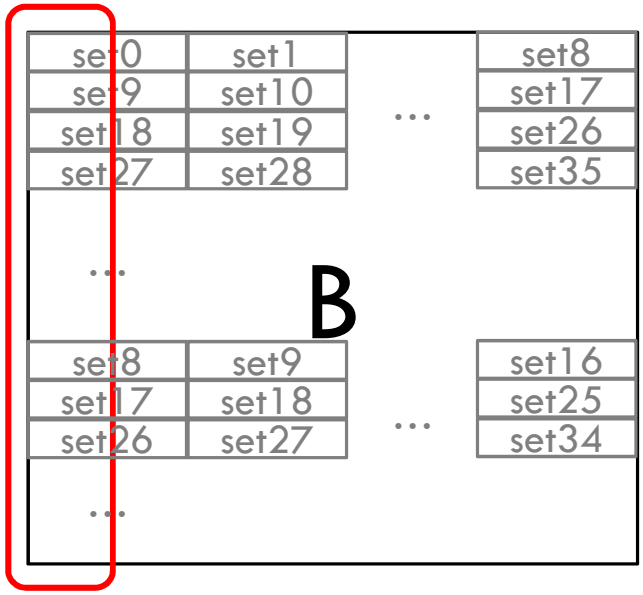
CACHE WAY CONFLICT (SET ASSOCIATIVITY!)

- If N is a multiple of 2^n
  - E.g., N=128
  - 128 * 4B = 512B = 8 cache lines
- Your cache almost always has 2^n sets
  - E.g., Cortex A78AE has 64KB L1D cache, 4 ways
  - 64KB / 4 ways / 64B line = 256 sets
- The column data only goes to sets with 8k index
- Now all your data crash into only 1/8 of the cache
- ~~16N < cache_size~~ → 16N < cache_size/8



	way 0	way 1	way 2	way 3
set 0	B[0][0]	B[32][0]	B[64][0]	B[96][0]
set 1	empty	empty	empty	empty
set 2	empty	empty	empty	empty
...				
set 7	empty	empty	empty	empty
set 8	B[1][0]	B[33][0]	B[65][0]	B[97][0]
set 9	empty	empty	empty	empty
...				
set 255	empty	empty	empty	empty

CACHE WAY CONFLICT (SET ASSOCIATIVITY!)

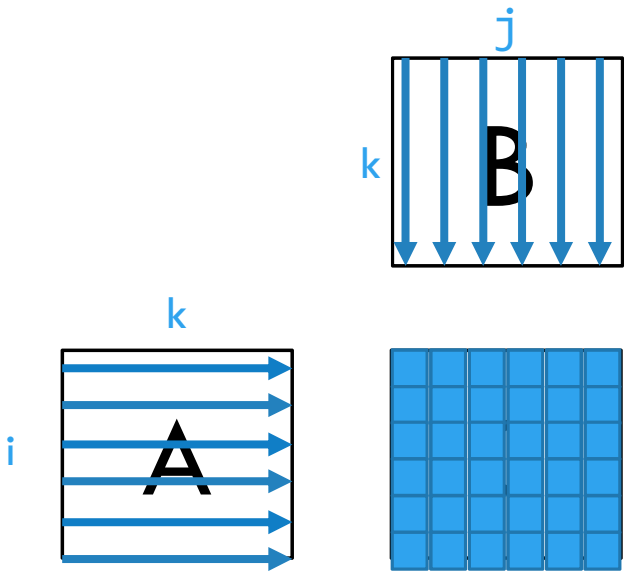


- When N/16 (number of cachelines) per row is <u>NOT</u> a multiple of 2^n
  - E.g., N=144
  - 144 * 4B = 576B = 9 cache lines

- The problem is gone. You can utilize the entire cache

	way 0	way 1	way 2	way 3
set 0	B[0][0]	...	...	...
...				
set 5	B[29][0]	...	...	...
...				
set 9	B[1][0]	...	...	...
...				
set 14	B[30][0]	...	...	...
...				
set 252	B[28][0]	...	...	...
...				

CACHE WAY CONFLICT

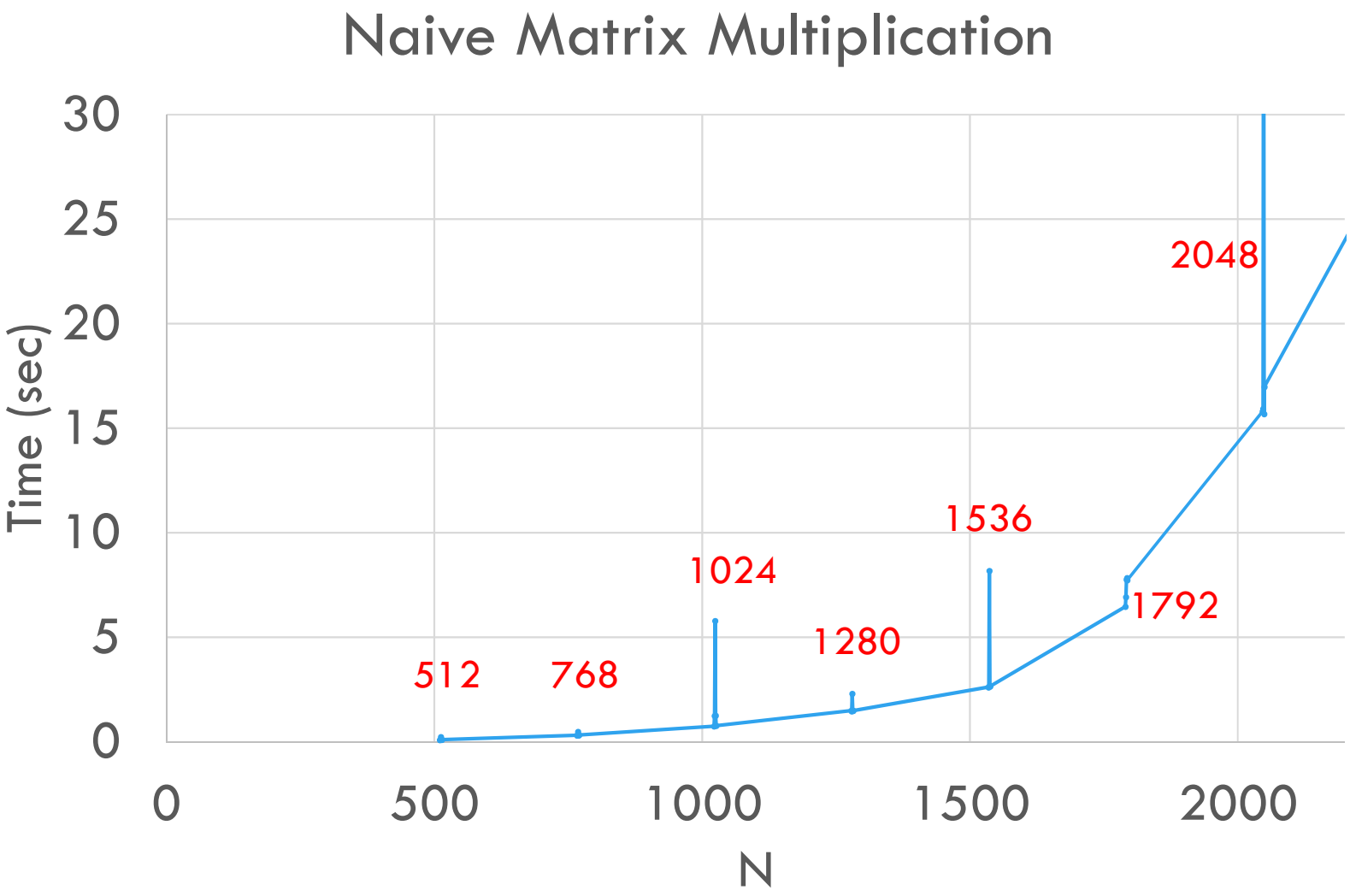



```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

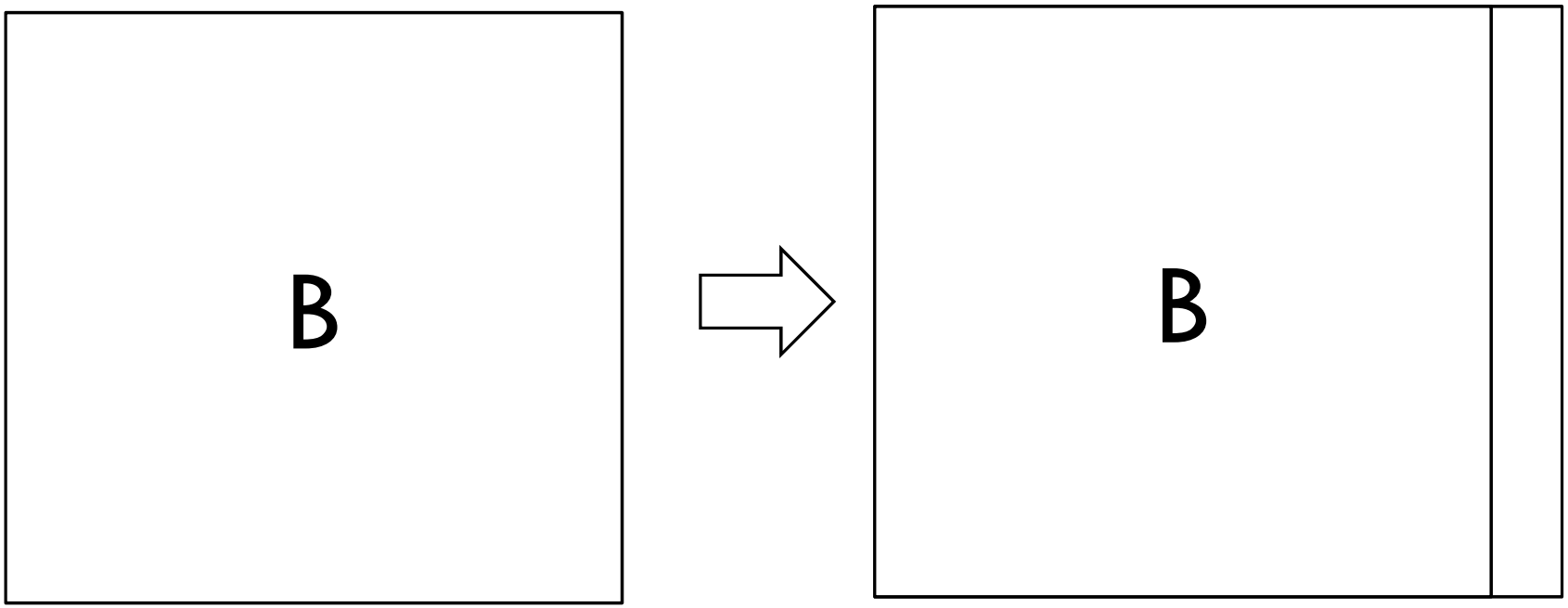
i	1→N
j	N*(1→N)
k	N^2*(1→N)

- A: N^2/c reads
- B: N^3/c reads (if 128N < cache size or 16N < cache if N is not a power of 2)
- C: N^2/c reads + N^2/c writes

LET'S MEASURE

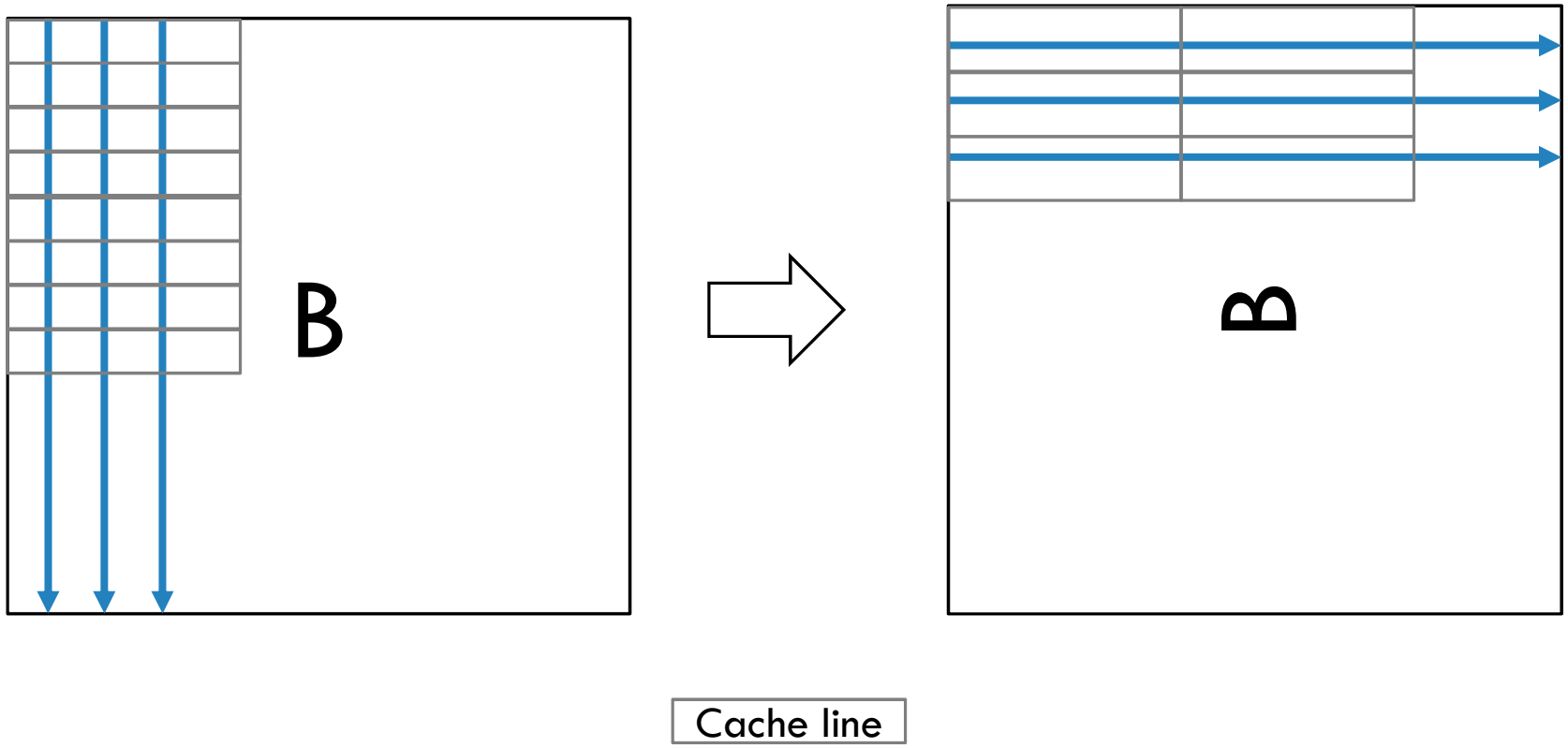


SOLUTION1: PADDING



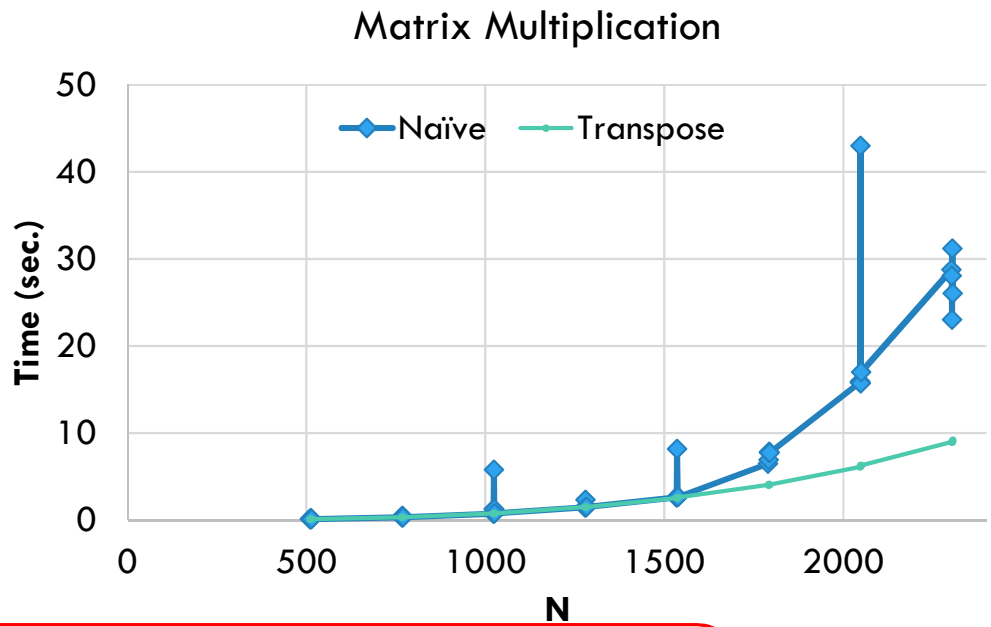
- Make a new array with a larger size, copy B into it
- You lose some capacity and bandwidth, but can get rid of the peaks at 2^N

SOLUTION2: TRANSPOSE



- Make a new array,
- Transpose the array before calculation
- Transpose itself takes additional N^2/c read + N^2/c write
- However it reduces $N^3 \rightarrow N^3/c$

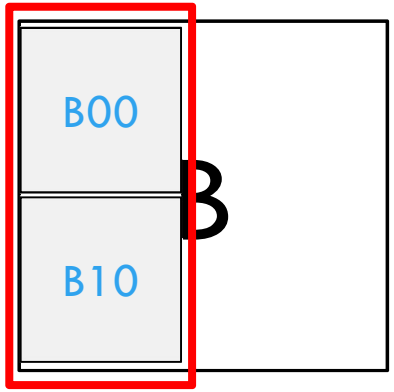
TRANSPOSE B + MATMUL



- A: N^2/c reads
- B: N^3/c reads (N^2 transpose cost is dwarfed)
- C: N^2/c reads + N^2/c writes

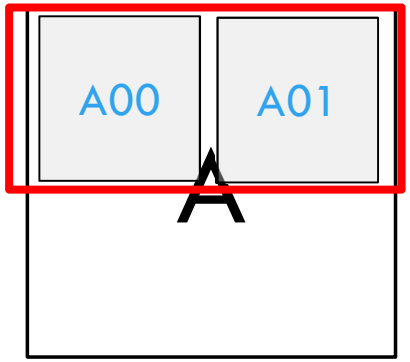
We finally got here.
Now we would like to reduce the N^3 part

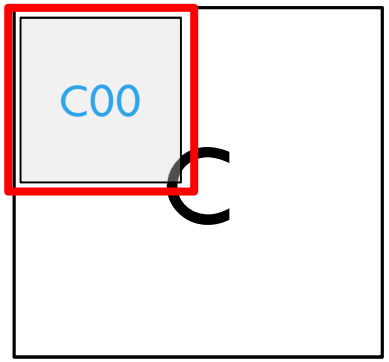
BLOCKED MATRIX MULTIPLY



- A kind of divide-and-conquer technique
- Divide the matrix so that it fits into the L1 cache

- $C_{ij} = \sum_k A_{ik} B_{kj}$

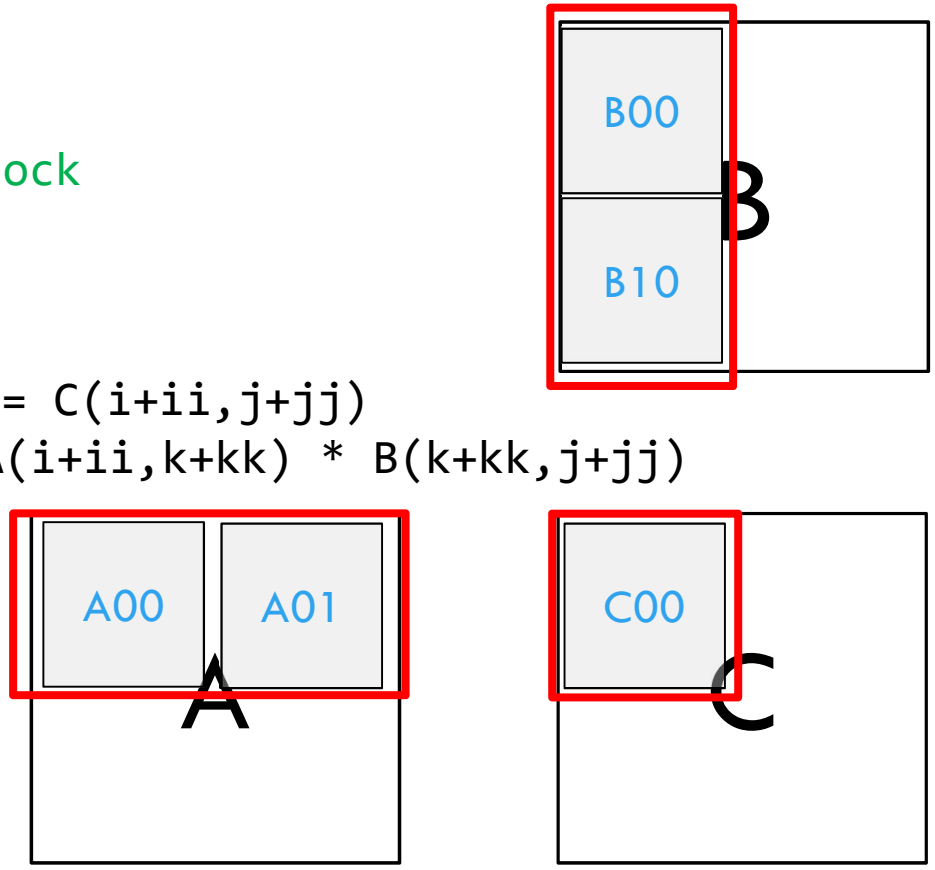




- Now A_{ik}, B_{kj}, C_{ij} are small matrices (blocks)

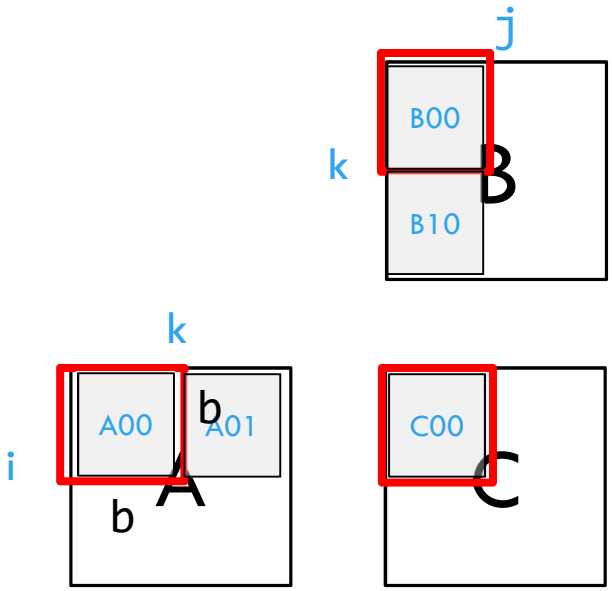
BLOCKED MATRIX MULTIPLY

```
for i = 1 to n step b
    for j = 1 to n step b
        for k = 1 to n step b
            //multiply the matrix block
            for ii = 1 to b
                for jj = 1 to b
                    for kk = 1 to b
                        C(i+ii,j+jj) = C(i+ii,j+jj)
                                     + A(i+ii,k+kk) * B(k+kk,j+jj)
```



- Now A_{ik}, B_{kj}, C_{ij} are small matrices (blocks)

BLOCKED MATRIX MULTIPLY



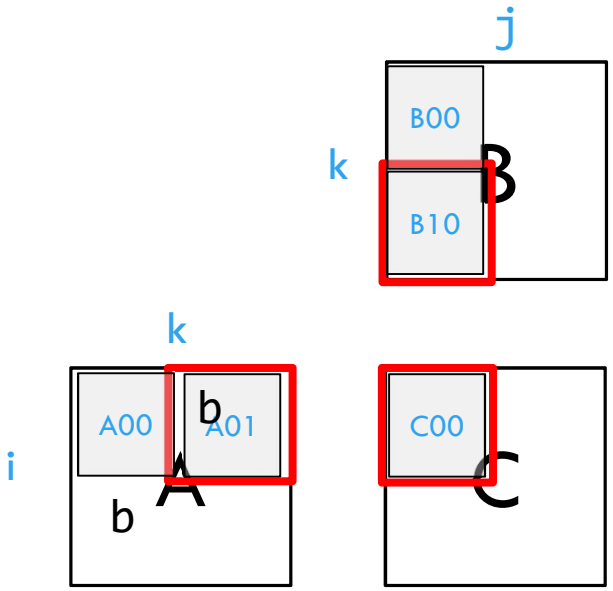

```
for i = 1 to n step b
    for j = 1 to n step b
        for k = 1 to n step b
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

This * is a block multiplication now

i	1
j	1
k	1

- Perform A00 * B00
  - Each block is a b*b matrix
  - A00 : b^2 / c reads
  - B00 : b^2 / c reads
  - Assume A00 and B00 fit in the cache

BLOCKED MATRIX MULTIPLY

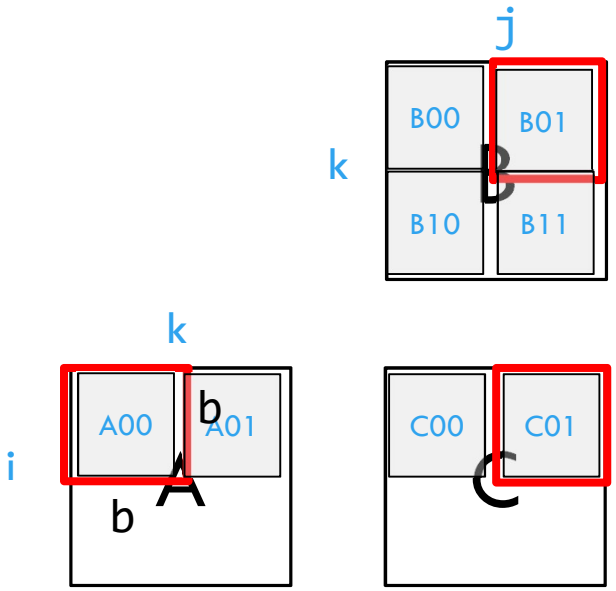



```
for i = 1 to n step b
    for j = 1 to n step b
        for k = 1 to n step b
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	1
k	2

- Perform A01 * B10
  - A01 : b^2 / c reads
  - B10 : b^2 / c reads
    - A01 and B10 fit in the cache

BLOCKED MATRIX MULTIPLY

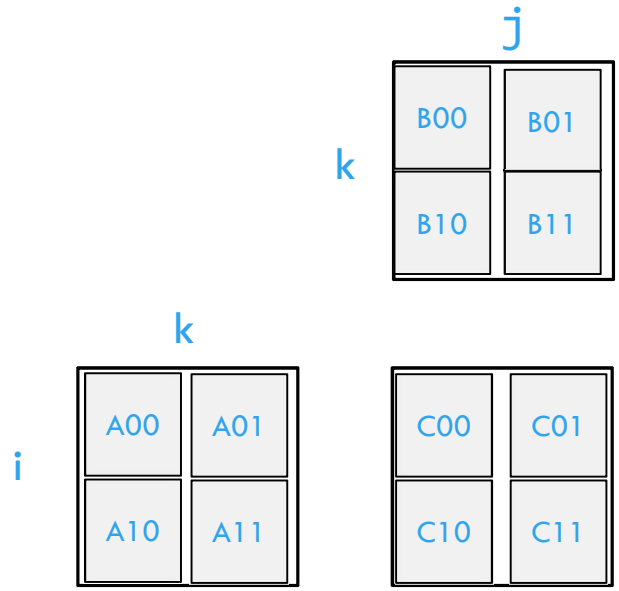



```
for i = 1 to n step b
    for j = 1 to n step b
        for k = 1 to n step b
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1
j	2
k	1

- Perform A00 * B10
  - A00 : b^2 / c reads
  - B10 : b^2 / c reads
  - This time, we do not expect entire A00 to be in the cache

BLOCKED MATRIX MULTIPLY

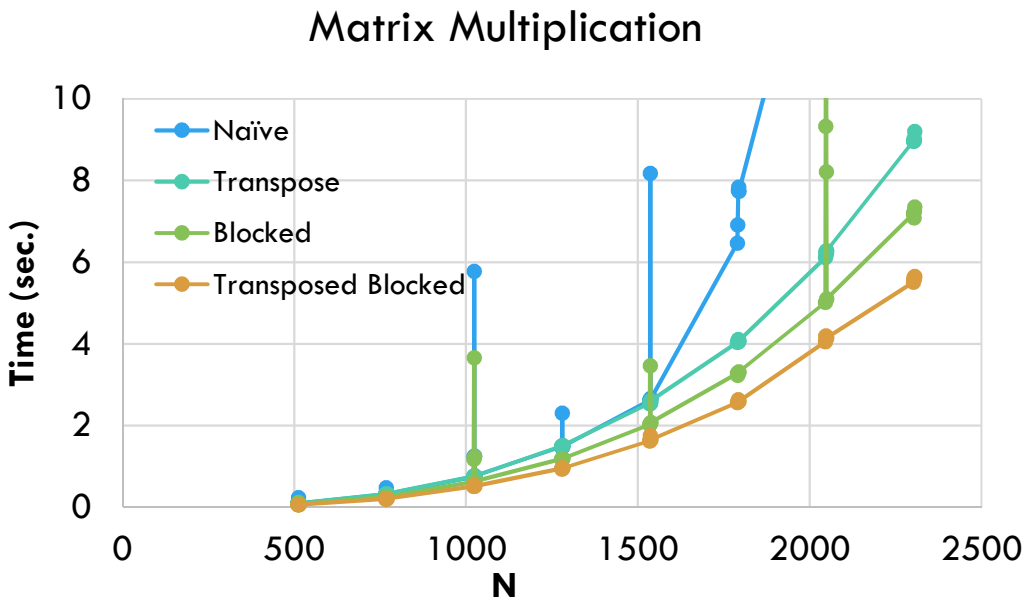



```
for i = 1 to n step b
    for j = 1 to n step b
        for k = 1 to n step b
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

i	1->N/b
j	(N/b)*1->N/b
k	$(N/b)^2$*1->N/b

- For the entire A*B
  - N/b: number of blocks
  - A: $(N/b)^3 * b^2 / c = N^3/bc =$ **$(N/b)\ N^2/c$** reads
  - B: $(N/b)^3 * b^2 / c = N^3/bc =$ **$(N/b)\ N^2/c$** reads
  - C: **N^2/c** reads and writes
- Larger b → faster matmul
  - Make sure b is set to fit the blocks in the cache!

BLOCKED MATRIX MULTIPLY



Blocked matmul

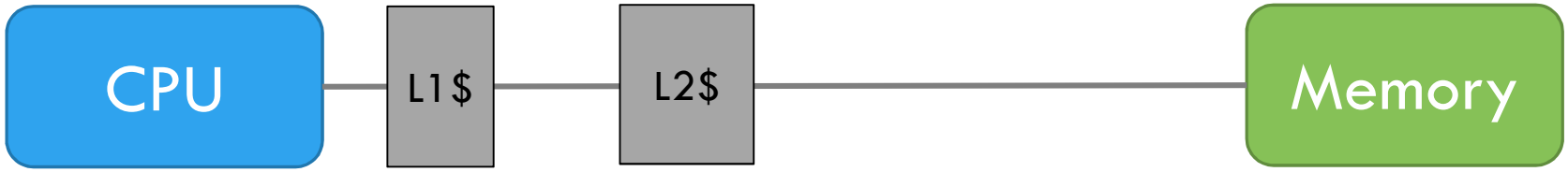
- A: **(N/b) N^2/c** reads
- B: **(N/b) N^2/c** reads
- C: **N^2/c** reads + **N^2/c** writes

Naïve matmul

- A: **N^2/c** reads
- B: **N^3/c** reads
- C: **N^2/c** reads + N^2/c writes

Q. Why do we still have those peaks at $N = 2^n$?

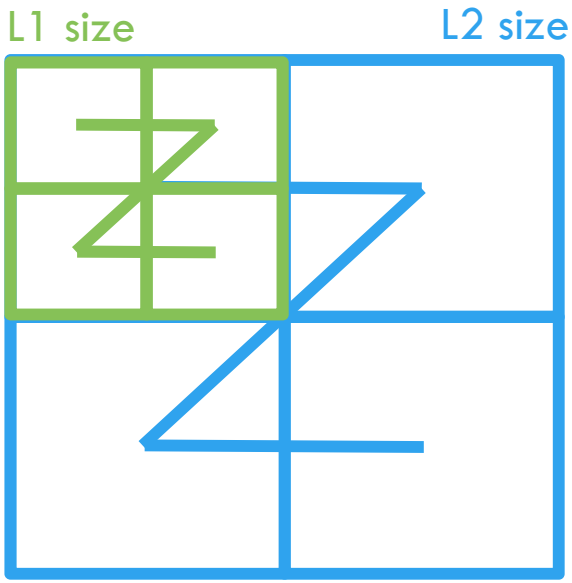
CONSIDER ANOTHER CACHE NOW



Small		Large
Low latency	← →	Long latency
High bandwidth		Low bandwidth

- We also have L2
- The same problem happens for L2 caches

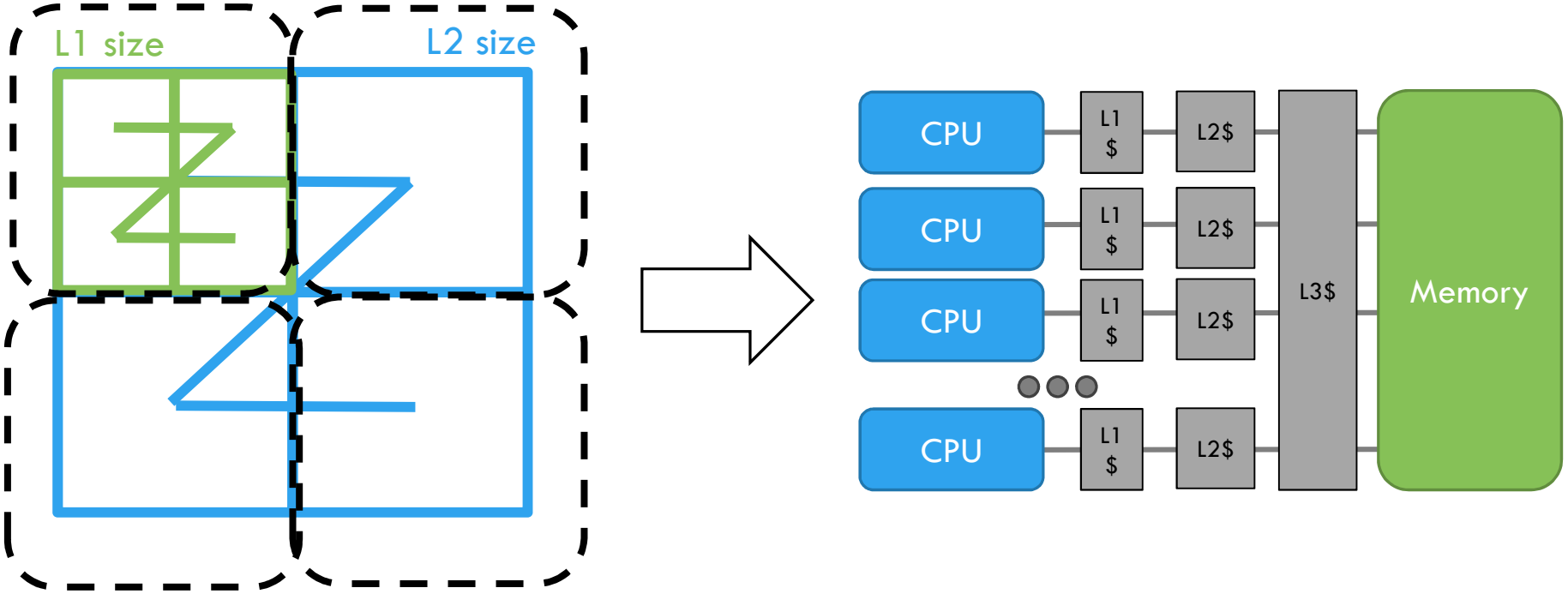
INTRODUCING TWO-LEVEL BLOCKING



- First block size: fit for L1
- Second block size: fit for L2

- Has a small additional improvement when N is large

INTRODUCING TWO-LEVEL BLOCKING



- No duplicate read to each core
- No duplicate computations

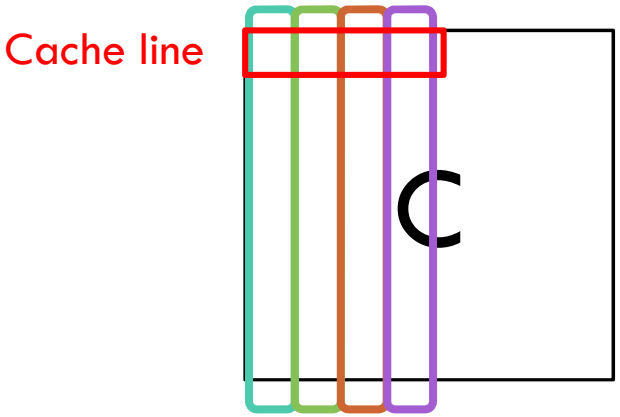
MORE THINGS YOU CAN TRY..

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
            C(i,j) = C(i,j) + A(i,k) * B(k,j)
```

- Loop reordering:
  - ijk -> jik / kji / ikj ...
- Loop unrolling
  - Compilers are bad at it
  - If you don't know about this, a great self-study topic
- Partition strategy
  - Block-strip
  - Block-column
  - By block – work queue
- Strassen/Winograd transform
  - Hard to parallelize, but possible!
- Vector processing (Neon, SSE, AVX, …etc)
  - Next lecture

PUTTING IT ALL TOGETHER

- Multithread + cache optimization
  - Leaves you a lot of options – Your Lab homework
- There's one more thing to consider: False sharing
  - Please have a flashback into the computer architecture course
- Even though no explicit communication,
- Coherence traffic occurs almost every time you write to C
  - Significant 'performance bug'
- Try to avoid this when partitioning the matrices



FINAL REMARKS

- Optimizing applications require a lot of effort, often depending on your specific hardware

- Using locks is bad, and there are other ways
  - Many times, write conflicts can be switched into read conflicts (which does not require locks)

- Matrix multiplication is used almost everywhere
  - Have fun!